

RAPPORT DE TEST

Projet de test et validation

NOM: KOUAKOU EBOUHO FRANCK YVAN

ENCADRANT: M.SEDRICK GAËL

PROJET: KOUAME AKA RICHARD

Sommaire

1. Introduction aux Tests Logiciels

- 1.1 Pourquoi tester un logiciel ?
- 1.2 Concepts clés
- 1.3 Assurance Qualité (QA) vs Test Logiciel
- 1.4 Tests manuels vs tests automatisés
- 1.5 Cycle de vie des tests logiciels
- 1.6 Types de tests
- 1.7 Conclusion

2. Stratégies et Techniques de Test

- 2.1 Techniques de test boîte blanche
- 2.2 Techniques de test boîte noire
- 2.3 Conception des cas de test
- 2.4 Rédaction d'un plan de test
- 2.5 Gestion des exigences et priorités
- 2.6 Travaux pratiques
- 2.7 Conclusion

3. Application au projet e-commerce

- 3.1 Acteurs du système
- 3.2 Architecture du projet
- 3.3 Exigences fonctionnelles
- 3.4 Cas de test détaillés
- 3.5 Inventaire des tests automatisés
- 3.6 Résultats des tests automatisés
 - 3.6.1 Tests unitaires
 - 3.6.2 Tests d'intégration
 - 3.6.3 Tests fonctionnels
 - 3.6.4 Tests surveillés (xfail)
- 3.7 Synthèse pytest
 - 3.7.1 Synthèse d'exécution
 - 3.7.2 Détails par test
- 3.8 Matrice de risque
- 3.9 Roadmap QA
- 3.10 Annexes
- 3.11 Conclusion

1. Introduction aux Tests Logiciels

1.1 Pourquoi tester un logiciel ?

- Détecter les bogues avant la mise en production (ex : panier ou commande).
- Améliorer la fiabilité et la qualité du parcours client.
- Réduire les coûts de maintenance en identifiant tôt les erreurs.
- Garantir la conformité aux spécifications fonctionnelles (vendeur, client, admin).

Exemple : un bug dans le calcul du total de commande peut générer des pertes pour les vendeurs.

1.2 Concepts clés

Terme	Définition	Exemple
Vérification	S'assurer que le logiciel est construit correctement.	<code>`Cart.get_total()`</code> calcule la somme exacte.
Validation	S'assurer que le logiciel répond aux besoins utilisateur.	Montant affiché au checkout conforme.

1.3 Assurance Qualité (QA) vs Test Logiciel

- Assurance Qualité : processus global (normes, méthodologie, sécurité).
- Test logiciel : activités ciblées pour détecter les défauts (pytest, contrôles manuels).

1.4 Tests manuels vs tests automatisés

Type de test	Avantages	Inconvénients
Manuels	Tests exploratoires, UX/UI	Lents, sensibles aux erreurs humaines
Automatisés	Rapides, répétables, CI/CD	Temps/maintenance initiaux

1.5 Cycle de vie des tests logiciels

- V-Model : planifier les tests dès la conception des modèles/vues.
- Agile / TDD : intégrer les tests à chaque itération (pytest).
- DevOps & CI/CD : automatiser les tests (pipeline GitHub Actions).

1.6 Types de tests

Activité	Description	Outils
Tests statiques	Analyse sans exécution	Revue code, flake8, bandit
Tests dynamiques	Exécution du code	pytest, unittest, Selenium

Type de test	Objectif	Exemple
Unitaires	Vérifier une fonction isolée	Tests modèles panier/commande
Intégration	Tester l'interaction modules	Flux checkout
Fonctionnels	Valider le comportement global	Scénario achat complet

1.7 Conclusion

Les principes sont appliqués en vérifiant chaque fonctionnalité critique de l'e-commerce (authentification, panier, commandes) et en combinant QA et tests automatisés.

2. Stratégies et Techniques de Test

2.1 Techniques de test boîte blanche

Type de couverture	Description	Adaptation projet
Instructions	Chaque ligne exécutée	Tests modèles (cart, order)
Branches	Chaque branche if/else	Validation des stocks et statuts
Conditions	Toutes les conditions logiques	Rôle utilisateur vs accès
Chemins	Combinaisons de chemins	Workflow commande + mise à jour statut

2.2 Techniques de test boîte noire

Technique	Description	Application projet
Partition d'équivalence	Groupes d'entrées similaires	Types d'utilisateur (client, vendeur, admin)
Analyse des valeurs limites	Tester les extrêmes	Quantités panier (0, 1,

		>stock)
Tables de décision	Combinaisons d'entrées	Autorisations / routes
Transitions d'état	Changements d'état système	Cycle commande pending → delivered

2.3 Conception des cas de test

ID	Cas de test	Entrées	Résultat attendu	Statut
TC01	Vérifier le total panier	Panier avec plusieurs items	Total correct	✓
TC02	Connexion avec identifiants valides	email/password	Redirection compte	✓
TC03	Connexion invalide	email / mauvais mot de passe	Message erreur	✓
TC04	Accès vendeur non autorisé	Visiteur → /vendeur	Redirection accueil	⚠

2.4 Rédaction d'un plan de test

- Objectif : valider l'authentification et la gestion des commandes.
- Périmètre : formulaires d'inscription/connexion, panier, checkout, espace vendeur.
- Critères de succès : connexion sécurisée, contrôle stock, statuts commande mis à jour.

2.5 Gestion des exigences et priorités

Fonctionnalité	Probabilité d'échec	Impact	Priorité	Remarques
Connexion utilisateur	Élevée	Critique	Haute	Bloque l'accès
Paiement (futur)	Moyenne	Très critique	Haute	Prévoir tests sécurisés
Page contact	Faible	Mineur	Moyenne	Améliore la confiance

2.6 Travaux pratiques

La suite pytest mise en place sert de travail pratique : elle couvre API Django, modèles et vues, conformément aux objectifs.

2.7 Conclusion

Les techniques boîte blanche et boîte noire sont combinées pour sécuriser le projet : couverture de code, tests d'intégration et matrice de priorisation guident les actions QA.

3. Application au projet e-commerce

3.1 Acteurs du système

Acteur	Description	Capacités attendues
Client	Acheteur final	Consulter catalogue, gérer panier, commander
Vendeur	Gestionnaire de produits	CRUD produits, suivre commandes
Administrateur	Superviseur	Valider vendeurs, modérer via Django Admin

3.2 Architecture du projet

Élément	Description	Référence
Backend	Django 5.2.7 (MVT)	`shop/`, `ecom_richard/`
Base de données	SQLite	`db.sqlite3`
Frontend	Bootstrap 5.3, templates	`shop/templates`
Utilisateur	`shop.models.User`	Rôles visitor/vendeur/admin

3.3 Exigences fonctionnelles

ID	Exigence	Acteur	Statut	Notes
REQ-AUTH-01	Inscription/connexion	Client/Vendeur	Partiel	Manque confirmation email
REQ-ROLE-03	Rôle vendeur attribué par admin	Admin	À corriger	Champ public
REQ-CART-	Gestion panier	Client	OK	Tests

04				automatisés
REQ-ORDER-05	Commande & stock	Client	OK	Décrément stock
REQ-CMS-07	Pages Contact/FAQ	Tous	Manquant	À Ajouter

3.4 Cas de test détaillés

ID	Cas de test	Entrées	Résultat attendu	Statut
TC-AUTH-001	Connexion valide	Identifiants corrects	Redirection home	✓
TC-AUTH-002	Connexion invalide	Mot de passe erroné	Message d'erreur	✓
TC-AUTH-003	Mot de passe oublié	Email enregistré	Lien reset envoyé	⚠

ID	Cas de test	Entrées	Résultat attendu	Statut
TC-CART-001	Ajout produit	qty=2, stock=10	Produit ajouté + compteur	✓
TC-CART-002	Stock insuffisant	qty>stock	Message alerte	✓
TC-CART-003	Checkout panier vide	Panier vide	Redirection panier	✓

ID	Cas de test	Entrées	Résultat attendu	Statut
TC-SELL-001	Accès dashboard vendeur	Vendeur connecté	Dashboard accessible	✓
TC-SELL-002	Visiteur vers espace vendeur	Visiteur	Redirection accueil	✓
TC-SELL-003	Mise à jour	POST statut	Statut modifié	✓

statut
commande

3.5 Inventaire des tests automatisés

Fichier	Nombre de tests	Chemin relatif
test_auth.py	6	tests\test_auth.py
test_cart_orders.py	4	tests\test_cart_orders.py
test_models.py	4	tests\test_models.py
test_pages.py	4	tests\test_pages.py
test_sample_data.py	2	tests\test_sample_data.py
test_seller.py	4	tests\test_seller.py

3.6 Résultats des tests automatisés

3.6.1 Tests unitaires

ID	Cas de test	Entrées	Résultat attendu	Statut
TU01	test_cart_total_updates	Panier multi produits	Total exact	✓
TU02	test_cart_item_subtotal	Article x2	Sous-total correct	✓
TU03	test_order_string_representation	Commande	Libellé complet	✓
TU04	test_order_status_choices_cover_expected_states	Enum statut	Valeurs complètes	✓

3.6.2 Tests d'intégration

ID	Cas de test	Entrées	Résultat attendu	Statut
TI01	test_add_product_to_cart_flow	POST ajout	Redirection + quantité	✓

		panier			
TI02	test_checkout_creates_order_and_decrements_stock	POST	Commande	✓	
		checkout	+ stock		
TI03	test_seller_can_update_order_status	POST	Statut	=	✓
		statut	confirmed		
		vendeur			
TI04	test_checkout_with_empty_cart_displays_error	Panier	Redirection	✓	
		vide	panier		

3.6.3 Tests fonctionnels

ID	Cas de test	Entrées	Résultat attendu	Statut
TF01	test_homepage_loads	GET /	200 OK + contenu	✓
TF02	test_product_list_accessible	GET /produits	Catalogue accessible	✓
TF03	test_registration_page_accessible	GET /inscription	Formulaire affiché	✓
TF04	test_login_success_redirects_home	POST login	Redirection home	✓

3.6.4 Tests surveillés (xfail)

ID	Cas de test	Entrées	Résultat attendu	Statut
TS01	test_public_registration_should_not_expose_user_type	GET /inscription	Champ user_type masqué	⚠
TS02	test_homepage_should_not_instantly_offer_become_seller_cta	GET /	CTA vendeur contrôlé	⚠
TS03	test_password_reset_route_available	reverse password_r	Route existante	⚠

TS 04	test_contact_page_accessible	eset GET /contact	Page disponible	⚠
TS 05	test_faq_page_should_exist	GET /faq	Page FAQ	⚠

3.7 Synthèse pytest

Résumé	Statistiques
18 passed, 5 xfailed, 1 xpassed in 14.24s	6 fichiers de test

3.7.1 Synthèse d'exécution

Indicateur	Valeur
Tests passés	18
Tests xfail	5
Tests xpass	1
Durée	14.24s

3.7.2 Détails par test

#	Fichier	Test	Statut	Détails
1	tests\test_auth.py	test_login_success_redirects_home	✓	-
2	tests\test_auth.py	test_login_invalid_credentials_shows_error	✓	-
3	tests\test_auth.py	test_public_registration_should_not_expose_user_type	⚠ (xfail)	-
4	tests\test_auth.py	test_homepage_should_not_instantly_offer_become_seller_cta	⚠ (xfail)	(auto-inscription)
5	tests\test_cart_orders.py	test_add_product_to_cart_flow	✓	-

6	tests\test_cart_orderders.py	test_add_product_to_cart_stock_guard	✓	-
7	tests\test_cart_orderders.py	test_checkout_creates_order_and_decrements_stock	✓	-
8	tests\test_cart_orderders.py	test_checkout_with_empty_cart_displays_error	✓	-
9	tests\test_models.py	test_cart_total_updates	✓	-
10	tests\test_models.py	test_cart_item_subtotal	✓	-
11	tests\test_models.py	test_order_string_representation	✓	-
12	tests\test_models.py	test_order_status_choices_cover_expected_states	✓	-
13	tests\test_pages.py	test_homepage_loads	✓	-
14	tests\test_pages.py	test_product_list_accessible	✓	-
15	tests\test_pages.py	test_registration_page_accessible	✓	-
16	tests\test_sample_data.py	test_create_sample_data_script_creates_baseline_users	✓	-
17	tests\test_sample_data.py	test_create_sample_data_script_populates_catalogue	✓	-
18	tests\test_seller.py	test_seller_dashboard_accessible_for_seller	✓	-
19	tests\test_seller.py	test_seller_dashboard_denied_for_visitor	✓	-
20	tests\test_seller.py	test_seller_can_update_order_status	✓	-
21	tests\test_seller.py	test_only_admin_should_assign_seller_role	⚠ (xpass)	-
22	tests\test_auth.py	test_password_reset_route_available	⚠ (xfail)	(Workflow de

	y				réinitialisation de mot de passe non configurer.) [91%]
23	tests\test_auth.py	test_contact_page_accessible	△ (xfail)		(Page contact manquante dans les URLs publiques.) [95%]
24	tests\test_pages.py	test_faq_page_should_exist	△ (xfail)		(La page FAQ n'est pas implementer.) [100%]

3.8 Matrice de risque

Fonctionnalité	Probabilité	Impact	Priorité	Action
Attribution rôle vendeur	Haute	Critique	1	Retirer champ + workflow
Reset mot de passe	Moyenne	Critique	2	Ajouter flux Django
Pages Contact/FAQ	Faible	Majeur	3	Créer pages dédiées
Couverture vues	Moyenne	Moyen	4	Compléter tests

3.9 Roadmap QA

Sprint	Livrables QA	Critères de succès
S+1	Workflow vendeur + reset	Tests xfail → pass
S+2	Pages Contact/FAQ	Navigation mise à jour
S+3	Couverture `shop.views`	> 80%

S+4

Audit sécurité + Selenium

Anomalies < 5

3.10 Annexes

Référence	Description	Statut
Rapport_Complet_Test_Validation_ECommerce.md	Rapport complet interne	À jour
Module 1	Introduction aux tests	Référencé
Module 2	Techniques avancées	Référencé
tests/	Suite pytest	Exécutée

3.11 Conclusion

le projet e-commerce est loin d'être achevé mais applique les enseignements : stratégie QA complète, tests automatisés et plan d'amélioration priorisé.